الجامعة الدولية للرباط
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵖⵔⴰⵡⵉⵜ ⵏ ⵔⵕⴱⴰⵟ
Université Internationale de Rabat
L'UNIVERSITÉ INNOVANTE

Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Cloud Foundations

Chapter 6-Part 2: Docker Containers

PR. OUASSIM KARRAKCHOU

UNIVERSITE INTERNATIONALE DE RABAT

Academic year: 2026-2027



Docker Containers

Docker Terminology

- Like many large efforts, the Docker project has spawned new terminology.
- In addition to tools used to create and launch an app, the extended environment includes tools used to deploy and control copies of running containers.

Based on what you did in the TP, can you cite some of these tools/commands?

- We will now examine some of the Docker terminology:
  - Image:
    - A Docker image is a file that can be executed, and a container is the execution of an image.



Docker Containers

Docker Terminology

- We will now examine some of the Docker terminology:
  - Partial Image:
    - An image can have two forms: a partial image that forms a building block, and a container image that includes all the software needed for a particular app.
    - Partial images form a useful part of the Docker programming environment analogous to library functions in a conventional programming environment.
    - That is, Docker offers a registry of pre-built partial images that each handle a common task.



Docker Containers

Docker Terminology

- We will now examine some of the Docker terminology:
  - Layer:
    - Docker uses the term layer to refer to each piece of code a programmer includes in a container image.
    - We say that the programmer starts by specifying a base image and then adds one or more layers of software.
    - For each layer, a programmer has a choice: the programmer can write code from scratch or download one of the pre-built building blocks from a registry.



Docker Containers

Docker Terminology

- We will now examine some of the Docker terminology:
  - Dockerfile:
    - Docker does not use conventional compiler tools to build an image. Instead, it uses a special build facility analogous to Linux's make.
    - A programmer creates a text file with items that specify how to construct a container image. By default Docker expects the specifications to be placed in a text file named Dockerfile.
    - A programmer runs docker build which reads Dockerfile, follows the specified steps, constructs a container image, and issues a log of steps taken (along with error messages, if any).



Docker Containers

Docker Terminology

- We will now examine some of the Docker terminology:
  - Container:
    - A Docker container image is not an executable file, and cannot be launched the same way one launches a conventional app.
    - Instead, one must use the command docker run to specify that an image is to be run as a Docker container.
    - A container is thus the execution of a complete container image.



Docker Containers

Docker Terminology Summary

Docker Term	Meaning	Analogous To
image	A file that contains binary code for a container along with its dependencies	a.out file (MacOS/Linux) .exe file (Windows)
layer	One piece of software added to an image as it is built	a software module
container	An instance of an image	an executing process
Dockerfile	A specification for how to build an image	Makefile (Linux)
docker build	A command that constructs an image according to a Dockerfile	"make" program (Linux)
docker run X	A command that runs image X as a container	launch app X



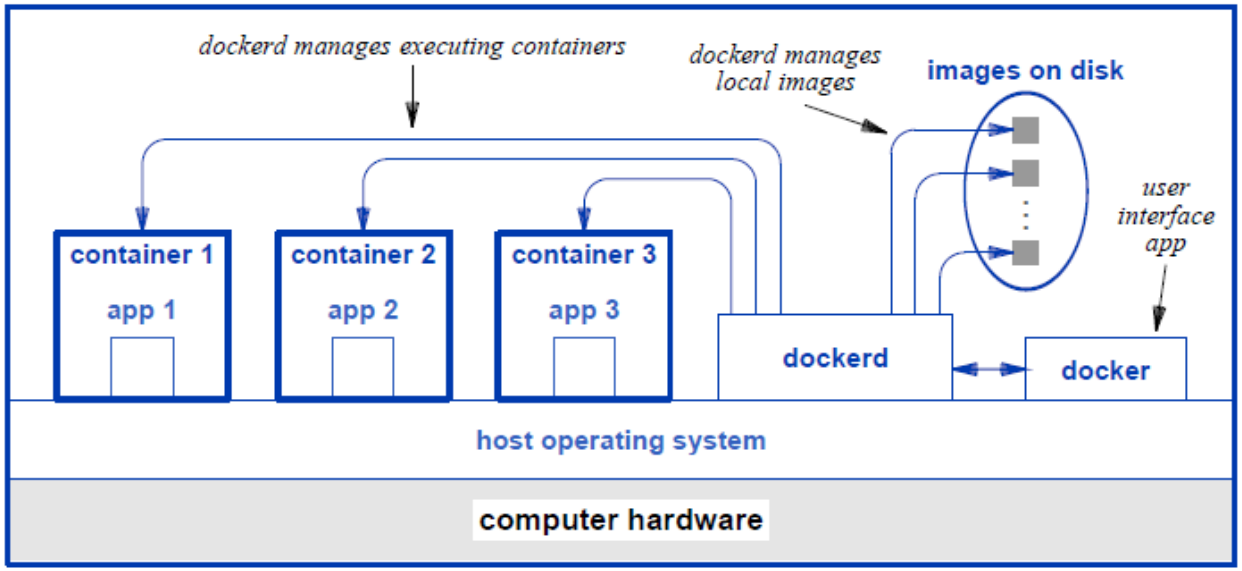
Docker Containers

Docker Software Components

- Execution of Docker containers requires additional software support.
- In particular, containers operate under the control of an application known as a Docker daemon (dockerd).
- In addition, Docker provides a user interface program, docker.
- Collectively, the software is known as the Docker engine.
- The dockerd program, which remains running in background at all times, contains several key subsystems. In addition to a subsystem that launches and terminates containers, dockerd contains a subsystem used to build images and a subsystem used to download items from a registry.

Docker Containers

Docker Software Components



- dockerd manages images that have been saved locally and running containers.

- Users do not interact with dockerd directly. Instead, dockerd provides two interfaces through which a user can make requests and obtain information:

  - A RESTful interface intended for applications
  - A Command Line Interface (CLI) intended for humans



Docker Containers

Dockerd Interfaces

- A RESTful interface intended for applications :
  - Customers of cloud services do not usually create and manage containers manually.
  - Instead, they use orchestration software to deploy and manage sets of containers.
  - When it needs to create, terminate, or otherwise manage containers, orchestration software uses dockerd's RESTful interface.
  - As expected, the RESTful interface uses the HTTP protocol, and allows orchestration software to send requests and receive responses.



Docker Containers

Dockerd Interfaces

- A Command Line Interface (CLI) intended for humans:
  - Customers of cloud services do not usually create and manage containers manually.
  - To accommodate situations when a human needs to manage containers manually, dockerd offers an interactive command-line interface that allows a user to enter one command at a time and receive a response.
  - When a user installs Docker software, the installation includes an application named docker that provides the CLI.



Docker Containers

Dockerd Interfaces

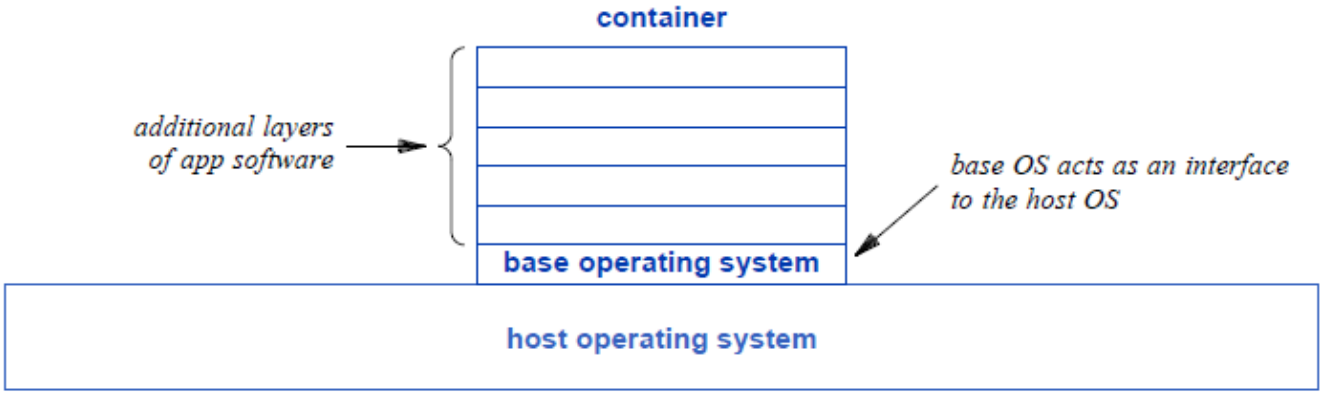
- A Command Line Interface (CLI) intended for humans:

Command	Meaning
docker build .	Read Dockerfile in the current directory and follow the directives to construct an image
docker images	List all images on the local machine
docker run	Create and start a container running a specified image
docker ps	List all currently running containers on the system
docker pull	Download a copy of an image from a registry
docker stop	Stop one or more running containers
docker start	Restart one or more stopped containers

Docker Containers

Base Operating System and Files

- In addition to layers of app software, most container images include a layer of software known as a base operating system that acts as an interface to the underlying host operating system.





Docker Containers

Base Operating System and Files

- Apps running in the container make calls to the base operating system which then makes calls to the host operating system.
- When building a container image, a programmer starts by specifying a base operating system; base operating systems are available in registries.
- From a programmer's perspective, most of the distinctions between a base operating system and the host operating system are unimportant. A programmer can use the base operating system to run commands or launch apps as usual.
- The apps make operating system requests as usual (e.g., to communicate over a network). If a user runs an interactive app in a container (e.g., a shell) and runs the container interactively, the app will connect to the user's standard input and output devices, just as if the programmer launched a conventional app.



Docker Containers

Base Operating System and Files

- One aspect of containers differs from conventional apps: the file system.
- When a container image is created, the base operating system includes a small file system that holds apps and other data.
- Apps running in the container can read files and can create new files.
- Because a container is immutable, however, files created when a container runs will not persist across invocations of the container image — each time a user creates a container to run a given image, the file system in the container will start with the files that were present when the image was initially created.
- Unless steps are taken, files created in the container will not persist once the container terminates.



Docker Containers

Dockerfile keywords

- A Dockerfile specifies a sequence of steps to be taken to build a container image.
- Each item in a Dockerfile begins with a keyword:
  - FROM : The FROM keyword specifies a base operating system to use for the image, must appear as the first keyword in a Dockerfile.
  - RUN : The RUN keyword specifies that a program should be run to add a new layer to the image. The name is unfortunately confusing because the "running" takes place when the image is built rather than when the container runs. For example, the programmer may run the apt program to install python in the container.



Docker Containers

Dockerfile keywords

- Each item in a Dockerfile begins with a keyword:
  - COPY : The COPY keyword copies files from the computer where the image is being built into the image file system.
  - ADD : The ADD keyword allows a programmer to specify local files or give a URL that specifies a remote file to be added to the container's file system. Furthermore, ADD understands how to decompress and open archives.
  - ENTRYPOINT : A programmer must specify where execution begins when a container starts running. To do so, a programmer uses the ENTRYPOINT keyword followed by the name of an executable program in the image file system and arguments that will be passed to the program.



Docker Containers

Dockerfile keywords

- Two keywords allow the container to communicate with the outside world:
  - EXPOSE : EXPOSE specifies protocol port numbers that the container is designed to use.
  - VOLUME :
    - VOLUME specifies a mount point in the image file system where an external file system can connect. That is, VOLUME provides a way for a container to connect to persistent storage in a data center.
    - VOLUME does not specify a remote storage location, nor does it specify how the connection to a remote location should be made. Instead, such details must be specified when a user starts a container.



Docker Containers

Dockerfile keywords summary

Item	Meaning
FROM	Specifies a base operating system for the image
RUN	Runs a program to fetch an item for the image
ENTRYPOINT	Specifies an app that runs when the container starts
CMD	Specifies a command that runs when the container starts
COPY	Copies local files into the image file system
ADD	An earlier form of COPY sometimes still seen
EXPOSE	Specifies network port numbers that the container will use
VOLUME	Specifies mount points for an external file system



Docker Containers

Dockerfile quizz

- What does this Dockerfile do?

```
FROM alpine
ENTRYPOINT ["/bin/echo", "hi there"]
```

```
PS C:\Users\ouassim.karrakchou\Desktop\test> docker build -t test .
[+] Building 29.7s (6/6) FINISHED
```

```
PS C:\Users\ouassim.karrakchou\Desktop\test> docker run test
Hi There
```



Containers

Summary

- *To construct a container image, a programmer invokes Docker's build mechanism, which follows the specifications in the Dockerfile and produces an image. Once it has been created, an image can be run in a container.*

- *Docker software includes two main pieces: a daemon named dockerd that runs in background, and a command named docker that a user invokes to interact with dockerd through a command-line interface.*

- *The docker interface allows a user to build an image or run and manage containers. Images are not stored in the user's directory; dockerd stores images and controls running containers.*

- *Dockerd also offers a RESTful interface that allows orchestration software to manage large-scale deployments of containers.*